



CyberVault: Secure File Encryption and Digital Signature System with Brute Force Protection and Secure Audit Logging



Prepared By: Ayesha Sajid

Registration ID: 247136

Department: BS CyberSecurity (4th Semester)

Institution: Air University Kharian Campus

- **Technology Used:** C++, OpenSSL, Kali Linux

Abstract

This project presents a secure file protection system using modern cryptographic techniques including AES, DES, RSA, DSA, and SHA-256. The system ensures confidentiality through encryption, integrity through hashing, and authenticity through digital signatures.

Additionally, cybersecurity features such as brute force attack prevention and secure audit logging are implemented to enhance system security. The project demonstrates secure file handling, tamper detection, and performance comparison between AES and DES algorithms.

Introduction

Cryptography is the science of securing communication and data from unauthorized access. This project implements both symmetric and asymmetric encryption techniques.

- AES and DES are used for symmetric encryption of files.
- RSA is used for secure key exchange.
- DSA is used for digital signatures to verify authenticity.
- SHA-256 is used for generating secure hash values.

The system also integrates cybersecurity mechanisms such as brute force protection and audit logging to simulate real-world secure systems.

Objectives

- Implement AES and DES encryption for files
- Use RSA for secure key exchange
- Implement digital signatures using DSA
- Generate SHA-256 hashes for integrity verification
- Prevent brute force attacks on authentication system
- Maintain secure audit logs of all activities
- Compare performance of AES vs DES

Theory

○ **Symmetric Encryption (AES & DES)**

Symmetric encryption uses a single key for both encryption and decryption. AES is modern, secure, and widely used, while DES is older and less secure.

○ **Asymmetric Encryption (RSA)**

RSA uses a public-private key pair. The public key encrypts data, and the private key decrypts it. It is used for secure key exchange.

○ **Digital Signatures (DSA/RSA)**

Digital signatures ensure data authenticity and integrity. They verify that the file has not been tampered with.

○ **Hash Functions (SHA-256)**

SHA-256 generates a unique fixed-length hash to detect file changes and ensure integrity.

6. Implementation

This system is implemented using:

- C++ for core logic
- OpenSSL for cryptographic operations
- Kali Linux for testing environment

Modules:

- AES/DES Encryption Module
- RSA Key Encryption Module
- DSA Signature Module
- SHA-256 Hash Module
- Brute Force Protection Module
- Secure Audit Logging Module

Project Directory Initialization

```
File Actions Edit View Help

(root@kali)-[~]
# /home/kali

(root@kali)-[/home/kali]
# mkdir CyberVault

(root@kali)-[/home/kali]
# cd CyberVault

(root@kali)-[/home/kali/CyberVault]
# mkdir keys

(root@kali)-[/home/kali/CyberVault]
# mkdir encrypted

(root@kali)-[/home/kali/CyberVault]
# mkdir decrypted

(root@kali)-[/home/kali/CyberVault]
# mkdir logs

(root@kali)-[/home/kali/CyberVault]
# mkdir hashes

(root@kali)-[/home/kali/CyberVault]
# mkdir signatures

(root@kali)-[/home/kali/CyberVault]
# mkdir temp

(root@kali)-[/home/kali/CyberVault]
#
```

Figure: Initializing the CyberVault Workspace

Using mkdir and cd, a central project directory is created followed by several sub-directories to organize keys, hashes, and encrypted data. This establishes a clean architectural baseline for the security system.

GUI Directory Verification

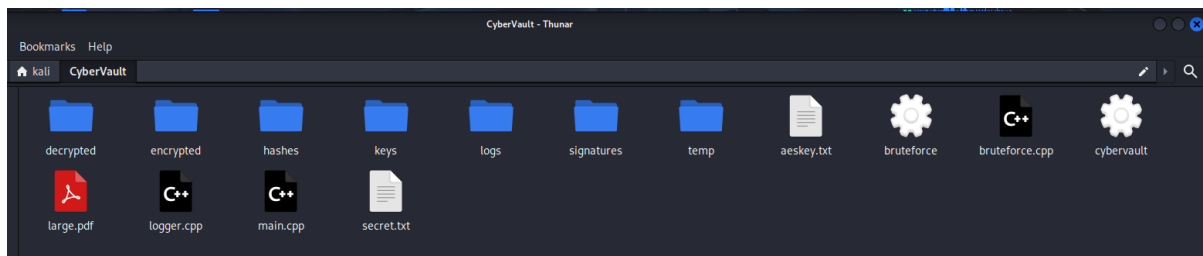


Figure: Visual Confirmation via Thunar File Manager

The Thunar file manager displays the folder, files hierarchy within CyberVault, confirming that the terminal commands were executed successfully. The folders and files decrypted, keys, and large are respectively ready to store their respective assets.

Full Project Directory Structure

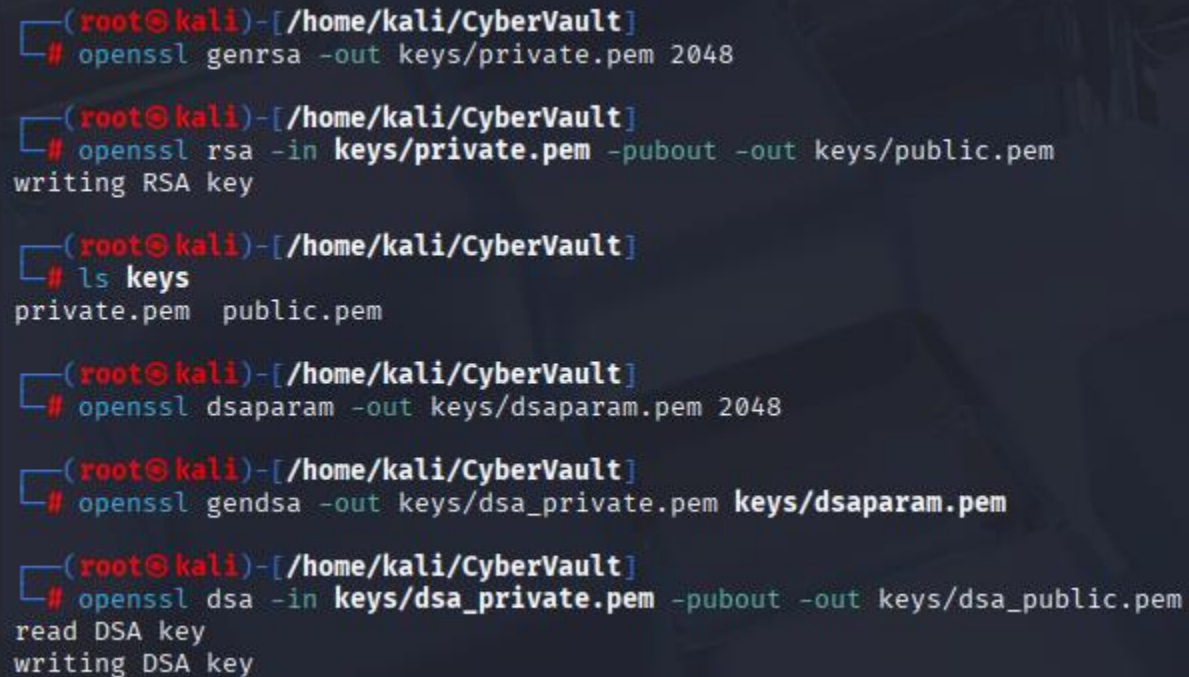
```
(root@kali)-[/home/kali/CyberVault]
# tree
.
├── aeskey.txt
├── bruteforce
├── bruteforce.cpp
├── cybervault
├── decrypted
│   ├── aeskey.txt
│   ├── des_secret.txt
│   └── secret.txt
├── encrypted
│   ├── aes.enc
│   ├── aeskey.enc
│   ├── des.enc
│   ├── des_secret.enc
│   ├── large_des.enc
│   ├── large.enc
│   └── secret.enc
├── hashes
│   └── filehash.txt
├── keys
│   ├── dsaparam.pem
│   ├── dsa_private.pem
│   ├── dsa_public.pem
│   ├── private.pem
│   └── public.pem
├── large.pdf
├── logger.cpp
├── logs
│   └── security.log
├── main.cpp
├── secret.txt
├── signatures
│   └── sign.bin
└── temp

8 directories, 26 files
```

Figure: Final State of the CyberVault Project

The tree command displays the final organized state of the project, now populated with 8 directories and 26 files. This view confirms the successful generation of cryptographic keys, encrypted data, security logs, and compiled C++ binaries.

RSA and DSA Key Generation



```
(root@kali)-[/home/kali/CyberVault]
# openssl genrsa -out keys/private.pem 2048

(root@kali)-[/home/kali/CyberVault]
# openssl rsa -in keys/private.pem -pubout -out keys/public.pem
writing RSA key

(root@kali)-[/home/kali/CyberVault]
# ls keys
private.pem  public.pem

(root@kali)-[/home/kali/CyberVault]
# openssl dsaparam -out keys/dsaparam.pem 2048

(root@kali)-[/home/kali/CyberVault]
# openssl gendsa -out keys/dsa_private.pem keys/dsaparam.pem

(root@kali)-[/home/kali/CyberVault]
# openssl dsa -in keys/dsa_private.pem -pubout -out keys/dsa_public.pem
read DSA key
writing DSA key
```

Figure: Generating Asymmetric Cryptographic Keys

OpenSSL is used to generate a 2048-bit RSA private key and its corresponding public key, followed by the creation of DSA parameters and keys. These keys form the foundation for secure encryption and digital signatures.

Source Code Creation and Compilation

```
(root@kali)-[/home/kali/CyberVault]
# vi main.cpp

(root@kali)-[/home/kali/CyberVault]
# cat main.cpp
#include <iostream>
using namespace std;
int main()
{
    cout << "CyberVault Security System" << endl;
    return 0;
}

(root@kali)-[/home/kali/CyberVault]
# g++ main.cpp -o cybervault

(root@kali)-[/home/kali/CyberVault]
# ./cybervault
CyberVault Security System

(root@kali)-[/home/kali/CyberVault]
# vi secret.txt

(root@kali)-[/home/kali/CyberVault]
# cat secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    return 0;
}
```

Figure: Developing the Main Application and Secret File

A C++ program is written to display a welcome message and compiled using g++, while a separate text file containing confidential information is created. This "secret" file serves as the primary target for the upcoming encryption tests.

AES-256 Symmetric Encryption

[illegible]

Figure : Encrypting and Decrypting with AES-256-CBC

The confidential file is encrypted into a non-readable format using AES-256 with a PBKDF2 iteration count of 100,000 for high security. The process is then reversed to verify that the original text can be successfully recovered.

Hashing and Digital Signatures

```
(root@kali)-[/home/kali/CyberVault]
# openssl enc -des-cbc -salt -in secret.txt -out encrypted/des_secret.enc
enter DES-CBC encryption password:
Verifying - enter DES-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(root@kali)-[/home/kali/CyberVault]
# openssl enc -des-cbc -salt -pbkdf2 -iter 100000 -in secret.txt -out encrypted/des_secret.enc
enter DES-CBC encryption password:
Verifying - enter DES-CBC encryption password:

(root@kali)-[/home/kali/CyberVault]
# openssl enc -des-cbc -d -pbkdf2 -iter 100000 -in encrypted/des_secret.enc -out decrypted/des_secret.txt
enter DES-CBC decryption password:

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 secret.txt
SHA2-256(secret.txt)= b743b2242fbf363bc5429be1f62073cbf01492aedd6634224854a27992dda070

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 secret.txt > hashes/filehash.txt

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -sign keys/dsa_private.pem -out signatures/sign.bin secret.txt

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -verify keys/dsa_public.pem -signature signatures/sign.bin secret.txt
Verified OK
```

Figure: Data Integrity and Authenticity Verification

A SHA-256 hash is generated to ensure file integrity, followed by the creation of a digital signature using the DSA private key. The public key is then used to verify the signature, confirming the file's authenticity.

Signature Verification Failure

```
(root@kali)-[/home/kali/CyberVault]
# vi secret.txt

(root@kali)-[/home/kali/CyberVault]
# cat secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    cout << "I Add this line to see Verification!!!" << endl;
    return 0;
}

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -verify keys/dsa_public.pem -signature signatures/sign.bin secret.txt
Verification failure
40F706A37D7F0000:error:030000EA:digital envelope routines:EVP_DigestVerifyFinal:provider signature failure:../crypto/evp/m_sigver.c:705:DSA digest_verify_final:
```

Figure: Detecting Unauthorized File Modifications

After the secret file is modified by adding an extra line, the signature verification process is re-run and fails. This demonstrates the system's ability to detect even minor changes in protected data, alerting the user to a potential breach.

Hybrid Encryption and Logger Development

```
(root@kali)-[/home/kali/CyberVault]
# openssl rand -base64 32 > aeskey.txt

(root@kali)-[/home/kali/CyberVault]
# openssl pkeyutl -encrypt -pubin -inkey keys/public.pem -in aeskey.txt -out encrypted/aeskey.enc

(root@kali)-[/home/kali/CyberVault]
# openssl pkeyutl -decrypt -inkey keys/private.pem -in encrypted/aeskey.enc -out decrypted/aeskey.txt

(root@kali)-[/home/kali/CyberVault]
# vi logs/security.log

(root@kali)-[/home/kali/CyberVault]
# vi logger.cpp

(root@kali)-[/home/kali/CyberVault]
# cat logger.cpp
#include <fstream>
#include <ctime>
using namespace std;
void writeLog(string message)
{
    ofstream log("logs/security.log", ios::app);
    time_t now = time(0);
    log << ctime(&now) << " : " << message << endl;
    log.close();
}
```

Figure: Asymmetric Key Wrapping and Logging Logic

OpenSSL is used to encrypt a symmetric AES key with an RSA public key, showcasing a hybrid encryption workflow. Additionally, a C++ logging function is developed to record security events with timestamps.

Brute Force Protection Simulation

```
(root@kali)-[/home/kali/CyberVault]
# vi bruteforce.cpp

(root@kali)-[/home/kali/CyberVault]
# cat bruteforce.cpp
#include <iostream>
#include <fstream>
#include <ctime>
#include <unistd.h>
using namespace std;
void writeLog(string message)
{
    ofstream logFile("logs/security.log", ios::app);
    time_t now = time(0);
    logFile << ctime(&now) << " : " << message << endl;
    logFile.close();
}
int main()
{
    string correctPassword = "1234";
    string enteredPassword;
    int attempts = 0;
    int maxAttempts = 3;
    while(attempts < maxAttempts)
    {
        cout << "\nEnter Password: ";
        cin >> enteredPassword;
        if(enteredPassword == correctPassword)
        {
            cout << "\nAccess Granted" << endl;
            writeLog("Successful Login");
            return 0;
        }
        else {
            attempts++;
            cout << "\nWrong Password!" << endl;
            writeLog("Failed Login Attempt");
            int remaining = maxAttempts - attempts;
            if(remaining > 0) {
                cout << "Remaining Attempts: " << remaining << endl;
            }
            sleep(5);
        }
    }
    cout << "\nSYSTEM LOCKED!" << endl;
    writeLog("System Locked Due To Multiple Failed Attempts");
}
```

Figure: Security Logic for Login Rate-Limiting

A C++ application simulates a login portal with a maximum of three attempts and a 5-second delay between failures. The program integrates with the logger to record successful logins, failed attempts, and system lockouts.

Brute Force Logic Execution

```
(root@kali)-[/home/kali/CyberVault]
# g++ bruteforce.cpp -o bruteforce

(root@kali)-[/home/kali/CyberVault]
# ./bruteforce

Enter Password: 1234

Access Granted

(root@kali)-[/home/kali/CyberVault]
# ./bruteforce

Enter Password: 4321

Wrong Password!
Remaining Attempts: 2

Enter Password: aish

Wrong Password!
Remaining Attempts: 1

Enter Password: kali

Wrong Password!

SYSTEM LOCKED!
```

Figure: Testing the Password Authentication and Lockout Mechanism

The compiled bruteforce binary is executed to demonstrate both a successful login and a multi-attempt failure scenario. Upon three incorrect entries, the program triggers a "SYSTEM LOCKED" state, simulating defensive countermeasures.

Security Audit Log Review

```
(root@kali)-[/home/kali/CyberVault]
# cat logs/security.log

Mon May 25 05:15:03 2026
: Successful Login
Mon May 25 05:15:10 2026
: Failed Login Attempt
Mon May 25 05:15:18 2026
: Failed Login Attempt
Mon May 25 05:15:26 2026
: Failed Login Attempt
Mon May 25 05:15:32 2026
: System Locked Due To Multiple Failed Attempts
```

Figure : Verifying Automated Event Logging

Using the cat command, the security.log file is inspected to confirm that all login attempts and system states were recorded with high-precision timestamps. This provides a definitive audit trail for monitoring unauthorized access attempts within the vault.

AES Performance Benchmarking

```
(root@kali)-[/home/kali/CyberVault]
# touch large.pdf

(root@kali)-[/home/kali/CyberVault]
# echo "This is a sample PDF content for testing." > large.pdf

(root@kali)-[/home/kali/CyberVault]
# time openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in large.pdf -out encrypted/large.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

real    5.84s
user    0.13s
sys     0.01s
cpu     2%
```

Figure: Timing AES-256-CBC Encryption on Large Files

The time utility is used alongside OpenSSL to measure the computational cost of encrypting a simulated PDF file using the AES-256 standard. This benchmark reveals the "real," "user," and "sys" time required to process secure data with 100,000 PBKDF2 iterations.

DES Performance Benchmarking

```
(root@kali)-[/home/kali/CyberVault]
# time openssl enc -des-cbc -salt -pbkdf2 -iter 100000 -in large.pdf -out encrypted/large_des.enc
enter DES-CBC encryption password:
Verifying - enter DES-CBC encryption password:

real    5.65s
user    0.07s
sys     0.00s
cpu     1%
```

Figure: Comparative Analysis of DES Encryption Speed

A similar performance test is conducted using the DES-CBC algorithm to compare its efficiency against the modern AES standard. While the execution times are recorded, this test highlights the performance-to-security trade-offs between different legacy and modern ciphers.

Compare Results

Algorithm	Encryption Time	Security
AES-256	Fast	Very Strong
DES	Faster	Weak

7. Results

The system successfully performs:

- File encryption using AES and DES
- Secure decryption of encrypted files
- RSA-based key encryption and decryption
- Digital signature generation and verification
- Tamper detection using hash comparison
- Brute force attack prevention
- Security event logging

Test Cases:

TEST 1 — AES Encryption & Decryption Test

```
(root@kali)-[/home/kali/CyberVault]
# cat secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    cout << "I Add this line to see Verification!!!" << endl;
    return 0;
}

(root@kali)-[/home/kali/CyberVault]
# openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in secret.txt -out encrypted/secret.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

(root@kali)-[/home/kali/CyberVault]
# cat encrypted/secret.enc
Salted__^ZAY
m[+b]TBF*6++="e++t?+}*6T+^,+++X+b+++[++++~+J++++a""+"h+++U3++++PE+$,9++1      >+++gA+d<4K+++i)0+I\++++1-w+++m++v{+++s++I+}++_>

(root@kali)-[/home/kali/CyberVault]
# openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -d -in encrypted/secret.enc -out decrypted/secret.txt
enter AES-256-CBC decryption password:

(root@kali)-[/home/kali/CyberVault]
# cat decrypted/secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    cout << "I Add this line to see Verification!!!" << endl;
    return 0;
}
```

Figure: AES File Encryption and Successful Decryption Output

This screenshot shows the encryption of a file using AES-256 algorithm and its successful decryption. It verifies that the original file content is securely restored without any data loss.

TEST 2 — DES Encryption & Decryption Test

```
(root@kali)-[/home/kali/CyberVault]
# openssl enc -des-cbc -pbkdf2 -iter 100000 -salt -in secret.txt -out encrypted/des_secret.enc
enter DES-CBC encryption password:
Verifying - enter DES-CBC encryption password:

(root@kali)-[/home/kali/CyberVault]
# cat encrypted/des_secret.enc

Salted__
U+00000000%e0000000,S
;T0000|yT09 ggF00C0Qs0

(root@kali)-[/home/kali/CyberVault]
# openssl enc -des-cbc -pbkdf2 -iter 100000 -d -in encrypted/des_secret.enc -out decrypted/des_secret.txt
enter DES-CBC decryption password:

(root@kali)-[/home/kali/CyberVault]
# cat decrypted/des_secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    cout << "I Add this line to see Verification!!!" << endl;
    return 0;
}
```

Figure: DES Encryption and Decryption Process Verification

This screenshot demonstrates file encryption using DES algorithm and successful recovery of original data after decryption, confirming functional symmetric encryption.

TEST 3 — SHA-256 Hash Test

```
(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 secret.txt
SHA2-256(secret.txt)= 81d4584e06018734a02a23a107f3008860d72f52e607dfcf1fa01fcb21fe1e24

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 secret.txt > hashes/filehash.txt

(root@kali)-[/home/kali/CyberVault]
# cat hashes/filehash.txt
SHA2-256(secret.txt)= 81d4584e06018734a02a23a107f3008860d72f52e607dfcf1fa01fcb21fe1e24

(root@kali)-[/home/kali/CyberVault]
#
```

Figure: SHA-256 Hash Generation for File Integrity

This screenshot displays the generated SHA-256 hash value of the file. It ensures data integrity by producing a unique fingerprint for file verification.

TEST 4 — Digital Signature Verification

```
(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -sign keys/dsa_private.pem -out signatures/sign.bin secret.txt

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -verify keys/dsa_public.pem -signature signatures/sign.bin secret.txt
Verified OK
```

Figure: DSA/RSA Digital Signature Verification Output

This screenshot shows successful verification of the digital signature, confirming the authenticity and integrity of the file before decryption.

TEST 5 — Tampering Detection Test

```
(root@kali)-[/home/kali/CyberVault]
# vi secret.txt

(root@kali)-[/home/kali/CyberVault]
# cat secret.txt
#include <iostream>
using namespace std;
int main()
{
    cout << "This is a Confidential CyberSecurity File!" << endl;
    cout << "Hacked!!!" << endl;
    return 0;
}

(root@kali)-[/home/kali/CyberVault]
# openssl dgst -sha256 -verify keys/dsa_public.pem -signature signatures/sign.bin secret.txt
Verification failure
4007464DFF7E0000:error:030000EA:digital envelope routines:EVP_DigestVerifyFinal:provider signature failure:../crypto/evp/m_sigver.c:705:DSA digest_verify_final:

(root@kali)-[/home/kali/CyberVault]
```

Figure: File Tampering Detection and Verification Failure

This screenshot demonstrates detection of file modification. The signature verification fails when the file content is altered, proving integrity protection.

TEST 6 — RSA Key Encryption & Decryption Test

```
(root@kali)-[/home/kali/CyberVault]
# openssl rand -base64 32 > aeskey.txt

(root@kali)-[/home/kali/CyberVault]
# openssl pkeyutl -encrypt -pubin -inkey keys/public.pem -in aeskey.txt -out encrypted/aeskey.enc

(root@kali)-[/home/kali/CyberVault]
# openssl pkeyutl -decrypt -inkey keys/private.pem -in encrypted/aeskey.enc -out decrypted/aeskey.txt

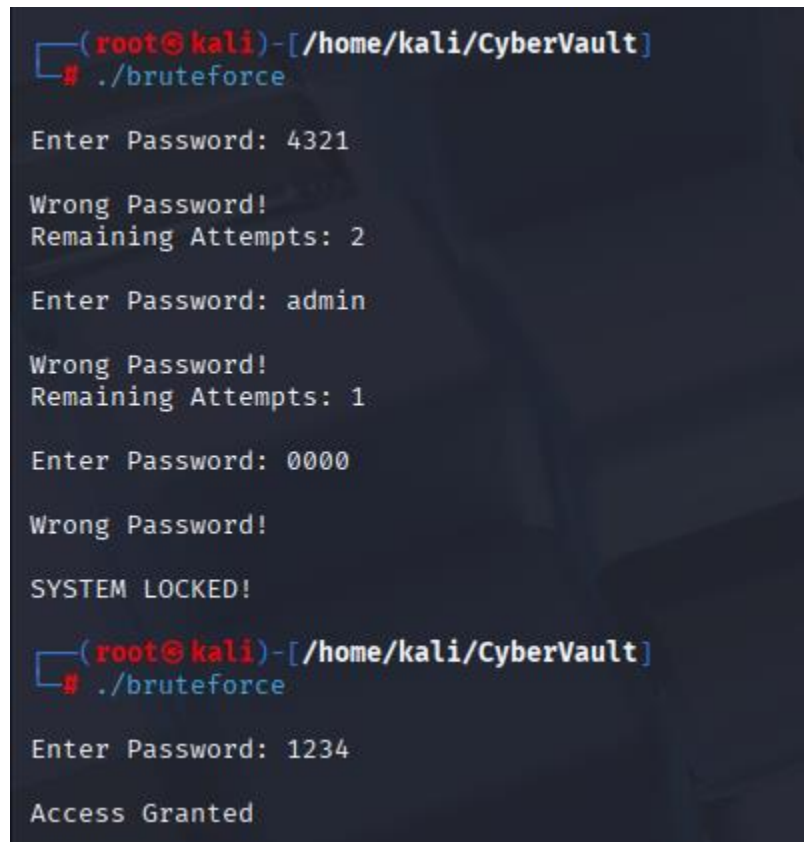
(root@kali)-[/home/kali/CyberVault]
# cat aeskey.txt
mKtHmmbU+E82v4LW/W8ZR0770BuUkEax72t2zDhLSGA=

(root@kali)-[/home/kali/CyberVault]
# cat decrypted/aeskey.txt
mKtHmmbU+E82v4LW/W8ZR0770BuUkEax72t2zDhLSGA=
```

Figure: RSA Encryption and Decryption of AES Session Key

This screenshot shows secure encryption of AES key using RSA public key and successful decryption using the private key, ensuring secure key exchange.

TEST 7 — Brute Force Protection Test



```
(root@kali)-[/home/kali/CyberVault]
# ./bruteforce

Enter Password: 4321

Wrong Password!
Remaining Attempts: 2

Enter Password: admin

Wrong Password!
Remaining Attempts: 1

Enter Password: 0000

Wrong Password!

SYSTEM LOCKED!

(root@kali)-[/home/kali/CyberVault]
# ./bruteforce

Enter Password: 1234

Access Granted
```

Figure: Brute Force Attack Prevention and System Lockout

This screenshot shows failed login attempts and system lockout after multiple incorrect password entries, demonstrating protection against brute force attacks.

TEST 8 — Secure Audit Logging Test

```
(root@kali)-[/home/kali/CyberVault]
# cat logs/security.log

Mon May 25 05:15:03 2026
: Successful Login
Mon May 25 05:15:10 2026
: Failed Login Attempt
Mon May 25 05:15:18 2026
: Failed Login Attempt
Mon May 25 05:15:26 2026
: Failed Login Attempt
Mon May 25 05:15:32 2026
: System Locked Due To Multiple Failed Attempts
Mon May 25 06:12:33 2026
: Failed Login Attempt
Mon May 25 06:12:38 2026
: Failed Login Attempt
Mon May 25 06:12:47 2026
: Failed Login Attempt
Mon May 25 06:12:52 2026
: System Locked Due To Multiple Failed Attempts
Mon May 25 06:12:59 2026
: Failed Login Attempt
Mon May 25 06:13:07 2026
: Failed Login Attempt
Mon May 25 06:13:21 2026
: Failed Login Attempt
Mon May 25 06:13:26 2026
: System Locked Due To Multiple Failed Attempts
Mon May 25 06:13:52 2026
: Successful Login
```

Figure: Security Audit Log Generation and Event Tracking

This screenshot displays security logs recording events such as encryption, failed login attempts, and system lockout, ensuring full activity tracking.

TEST 9 — AES vs DES Performance Comparison

```
(root@kali)-[/home/kali/CyberVault]
# time openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in large.pdf -out encrypted/aes.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

real    6.03s
user    0.11s
sys     0.01s
cpu     2%

(root@kali)-[/home/kali/CyberVault]
# time openssl enc -des-cbc -pbkdf2 -iter 100000 -salt -in large.pdf -out encrypted/des.enc
enter DES-CBC encryption password:
Verifying - enter DES-CBC encryption password:

real    7.19s
user    0.06s
sys     0.00s
cpu     0%

(root@kali)-[/home/kali/CyberVault]
#
```

Figure: Performance Comparison Between AES and DES Algorithms

This screenshot compares encryption time of AES and DES algorithms, showing AES as more secure while DES being faster but less secure.

Analysis

AES vs DES Performance

AES is more secure and widely used in modern systems, while DES is faster but insecure due to its small key size.

- AES provides higher security with minimal performance impact
- DES is faster but vulnerable to brute force attacks

Security Observations

- Brute force protection successfully blocks unauthorized access
- Audit logs provide complete system activity tracking
- Digital signatures effectively detect file tampering
- SHA-256 ensures strong integrity verification

9. Conclusion

This project successfully demonstrates a secure cryptographic system integrating encryption, hashing, and digital signatures. The addition of cybersecurity features such as brute force protection and audit logging enhances system reliability and real-world applicability.

The project provides practical understanding of cryptography and cybersecurity principles including confidentiality, integrity, authentication, and access control.
